



UNIVERSIDADE DA BEIRA INTERIOR
Engenharia

Geração Automática de Teste Unitário com Cobertura de Código Otimizada Integração no Frama-C

Nuno Miguel Apolinário Leitão

Dissertação para obtenção do Grau de Mestre em
Engenharia Informática
(2º ciclo de estudos)

Orientador: Prof. Doutor Simão Melo de Sousa

Covilhã, Junho de 2014

PREVIEW

Agradecimentos

O sucesso no desenvolvimento desta dissertação, bem como noutros aspetos de desenvolvimento pessoal, apenas foi possível dado a contribuição e ajuda prestada por diversas pessoas que, direta ou indiretamente, intervieram de forma muito positiva neste processo.

O meu maior agradecimento vai para os meus pais, Maria João e Manuel Leitão, e irmã, Mariana Leitão, que incansavelmente me aturam todos os dias, deram-me bons exemplos e apoiam-me sempre nas minhas escolhas. Graças a eles hoje estou aqui.

Existem ainda mais dois grandes modelos que também sempre acreditaram em mim: os meus avós maternos, Isabel Leitão e José Apolinário. Preocupados por natureza e ainda que com poucos conhecimentos, tentam sempre que faça boas opções na vida, dão tudo o que têm e não têm para que tenha sucesso. Todos os dias dão um pouco de si para tornar o dia de outra pessoa melhor, tudo o que fazem é sempre com vontade e carinho.

Deixo ainda um grande agradecimento aos restantes familiares que sempre me deram apoio e ajudam quando necessário.

Existe ainda um grupo que me acompanhou ao longo de todo o ano e que todas as semanas permitia usufruir uma hora e trinta minutos de pura diversão, relaxamento e convívio: o grupo de futebol das quintas-feiras às 19h, e todos os seus elementos que, incansavelmente, jogaram semana após semana sempre com vontade e boa disposição.

Uma das pessoas que também me ajudou a crescer pessoalmente, um grande amigo de infância, Mário Pereira, e que eventualmente voltei a encontrar na Universidade. Já várias vezes lhe agradei e vou voltar a fazê-lo pois foi uma pessoa que marcou a minha vida pessoal e académica.

Desde que entrámos para a Universidade sempre tive o seu apoio e, inicialmente, foi ele que me encaminhou para a vertente de investigação e que acabou por me apresentar o orientador de projeto de licenciatura e de dissertação de mestrado, o Professor Doutor Simão de Sousa.

O Professor Doutor Simão Melo de Sousa, orientador desta dissertação, foi o maior contributo direto para o desenvolvimento desta. Como profissional de investigação e mentor de diversos projetos, manifestou-se uma pessoa deveras incansável e sempre disponível, estando em todos os momentos receptível a prestar ajuda e/ ou esclarecimentos diversos, mesmo em períodos em que a quantidade de trabalho limitava em muito a sua disponibilidade.

Quero ainda agradecer ao Professor Doutor Simão Melo de Sousa pela oportunidade que me deu de lecionar aulas de formação. Foi um grande contributo para o meu desenvolvimento pessoal. Lecionar aulas revelou-se uma paixão, algo que adorei fazer e muitas das vezes, bastava umas horas de formação para sair da sala com um sorriso. Obviamente que para se gostar de lecionar aulas também é preciso ter excelentes pessoas como alunos, deixando também o meu agradecimento aos alunos do curso TPSI e TPSI1 pelas excelentes pessoas que são.

Agradeço ainda ao Joel Carvalho, Rui Braz e ao João Oliveira. Foi e é um enorme prazer trabalhar com eles pois permitiram-me crescer partilhando as suas experiências e conhecimentos na área da formação profissional.

Não posso deixar também de referir o Professor Doutor Paul Crocker e o Professor Doutor João Paulo Fernandes que, embora não tenham contribuído diretamente para o desenvolvimento da dissertação, mostraram-se sempre dispostos a esclarecer qualquer questão ou a prestar apoio.

Esta dissertação foi realizada no laboratório *RELIable And SEcure Computation Group (RELEASE)*. Foi um enorme prazer trabalhar na companhia destas pessoas que constituem este grande grupo de investigação. A todas elas o meu obrigado.

Existem ainda alguns amigos que diariamente me dão um pouco do seu dia para melhorar o meu: Nuno Pereira, Diogo Tavares, Nuno Santos, Eunice Martins e Christophe Soares.

Gostaria de agradecer ao Nuno Pereira a sua boa disposição diária que alegra o dia de qualquer pessoa, estando sempre disposto a ajudar, independentemente das horas ou situação. Não posso deixar de referir as maravilhosas aulas de Zumba, Step e Yoga para o qual me aconselhou a ir e certamente não me arrependo.

O Diogo Tavares, também um grande amigo de longa data, com o qual posso sempre contar. Embora agora esteja em Lisboa, é aquela pessoa com a qual se pode sempre falar ou pedir qualquer coisa.

O Nuno Santos, outro grande amigo de longa data, o rapaz do desporto e da guitarra! É um grande exemplo que as pessoas podem mudar. Tornou-se uma motivação diária para um estilo de vida saudável.

A Eunice Martins, uma amizade relativamente recente mas que proporciona momentos diários de descontração durante as horas de trabalho, momentos em que também é necessário ter uma conversa sem sentido e rir um pouco da situação. Uma pessoa sempre com boa disposição, vontade de ajudar e aprender.

O Christophe Soares, o homem normalmente desaparecido, mas uma excelente pessoa para ter conversas sérias e também sempre disponível a ajudar.

Gostaria de deixar ainda um breve agradecimento ao André Neves, grande amigo de longa data, considerado um irmão, que sabe muito bem o impacto que teve na pessoa que hoje sou.

Houve ainda uma pessoa, que embora tenha ido para outra Universidade, foi uma das grandes amizades que surgiu nestes anos, o Filipe Oliveira. Aquela pessoa sempre disposta a opinar sobre qualquer assunto com conselhos deveras honestos e fundamentados. Uma excelente pessoa com grandes ambições e potencial.

Agradeço ainda ao João Casteleiro, atual presidente do Núcleo de Informática da UBI (NINF), pela oportunidade de me deixar ingressar como elemento da direção. Foi, não só uma novidade, como me permitiu adicionar algumas características à minha lista de competências. Não posso deixar de agradecer também a todos os elementos que fizeram parte desta direção com a qual partilhei várias horas semanais em prol do desenvolvimento do núcleo.

Podia continuar a agradecer a todas as pessoas que contribuíram para esta dissertação e/ ou crescimento pessoal, mas essa lista seria infundável.

Embora possa haver pessoas que aqui não estejam mencionadas, elas certamente influenciaram em algum ponto fulcral durante estes anos, sejam os pequenos cafés para descontrair, as saídas à noites, as jogatanas de ténis, ou simples conselhos, ajudas ou conversas, deixando também a essas pessoas o meu sincero agradecimento.

Resumo

O teste de *software* é o principal modo de verificar a correção deste. Normalmente constitui 50% do seu custo de desenvolvimento. Existem diversas estratégias de teste de software mas, nesta dissertação, é essencialmente abordado o teste unitário, uma forma de teste que permite a análise de pequenos componentes individuais do sistema (funções, classes ou módulos). O seu principal objetivo é detetar erros lógicos nos componentes, verificar *corner cases* e tentar obter a maior cobertura de código testado possível. O grande problema desta abordagem é ser, normalmente, um processo manual, o que por sua vez o torna dispendioso, propício a erros e bastante demorado. Portanto seria uma mais valia conseguir automatizar o processo de geração e execução do teste unitário com uma elevada cobertura de código.

Nesta dissertação é apresentado e explorado o conceito de testes *Concolic* com o intuito de permitir ultrapassar algumas das limitações anteriormente mencionadas. O teste *Concolic* junta duas técnicas de teste de *software*: teste com valores concretos e simbólicos simultaneamente. Atualmente existem várias ferramentas que permitem realizar este tipo de teste de *software* e esta dissertação inspira-se sobre uma delas.

O principal foco desta dissertação foi a exploração e possível integração de uma ferramenta de teste *Concolic* no *Frama-C*. O *Frama-C*, implementado em *Objective Caml (Ocaml)*, disponibiliza um conjunto de ferramentas dedicadas a análise de código C. Fornece várias técnicas de análise estática numa única *framework* e é usado atualmente por empresas como *Airbus* e *Dassault Aviation*.

Deste modo foram analisadas e estudadas um conjunto de ferramentas que permitissem realizar teste *Concolic*. Posteriormente foi escolhida uma ferramenta (*Crest*) com o intuito adaptar os seus processo à plataforma *Frama-C* e à respetiva implementação em *Ocaml*. Esta ferramenta permite a execução de testes *Concolic* de programas desenvolvidos na Linguagem C e contempla várias etapas:

1. Inicialmente é feita automaticamente a instrumentação do código, isto é, altera-se o código original para um código que realize chamadas a funções que permitam a sua execução simbólica. Se o programa for fornecido com valores concretos, este produz o mesmo output que o original. Esta etapa é feita através do *Common Intermediate Language (CIL)*.
2. Finda a instrumentação é feito um ciclo de execução em que são armazenadas um conjunto de restrições das várias condições encontradas. Estas condições são armazenadas e posteriormente resolvidas, fazendo uso do *Yices*, e assim gerar novos valores para, na próxima execução, percorrer diferentes caminhos.

O resultado final desta dissertação contempla a adaptação dos principais fundamentos do *Crest* a uma implementação *Ocaml* e respetiva integração em *Frama-C*. Todas as etapas foram implementadas desde as várias etapas da instrumentação à execução dos testes, geração automática de inputs e integração no *Frama-C*.

Palavras-chave

Teste *Concolic*, Teste *Software*, Frama-C, Execução Simbólica, Execução Concreta, Pesquisa de *Bugs*, Teste Unitário.

PREVIEW

Abstract

Software testing is the main mechanism for verifying the soundness of any software. There are several strategies for doing it. This thesis focuses, primarily, on unit testing, which allows the verification of the smallest components of the system (functions, classes or modules). The main objective of this type of test is to detect logical errors in those components, to verify corner cases and to try to obtain the largest coverage of tested code as possible. However, there are problems associated. Usually, the biggest issue involved is that this type of testing has to be done manually, which makes the process expensive, error prone and time consuming. Therefore, it would be extremely useful to automatize the generation and execution process of unit testing.

This thesis will present and explore the concept of unit testing with the purpose of allowing to overcome some of the above mentioned limitations of this activity. Concolic testing establishes a connection between two software testing techniques: the testing with concrete and symbolic values, simultaneously. Currently, there are several tools available which allow this type of testing and this thesis is inspired in one of these tools.

The main focus of this thesis was the exploration and possible integration of a Concolic testing tool in Frama-C. The Frama-C, implemented in OCaml, offers a range of tools dedicated to the analyse of C code. It provides several analysis techniques in one framework and it's used nowadays by companies such as Airbus and Dassault Aviation.

Thus, a set of tools that make it possible the execution of Concolic tests was studied and analysed. Later, Crest was chosen as a reference implementation of a Concolic test tool with the purpose of implementing it in OCaml and to integrate in Frama-C. The latter makes it possible the execution of Concolic tests of programs developed in C language and contemplates several stages:

1. Initially, we proceed to the code instrumentation, which means that the original code is changed for another one that performs calls to functions that allow the symbolic execution. If the program is furnished with concrete values, this produces the same output as the original does. This step is done using Common Intermediate Language (*CIL*).
2. When the instrumentation process ends, a new cycle of execution is done, where a whole set of constraints of the conditions that the code may contain are stored and solved, through Yices, to make it possible the generation of new values and go through different paths in the next execution.

The final result of this thesis contemplates the adaptation of the main fundamentals of Crest to an OCaml implementation and its respective integration in Frama-C. All the stages were implemented from the various instrumentation steps until tests execution, automatic generation of inputs and integration in Frama-C.

Keywords

Concolic Testing, Software Testing, Frama-C, Symbolic Execution, Concrete Execution, Bug Finding, Unit Testing.

PREVIEW

Índice

1	Introdução	1
1.1	Motivação e Objetivos	1
1.2	Contribuições	2
1.3	Organização do Documento	3
2	Engenharia de Teste	5
2.1	Teste de Software	5
2.1.1	Estratégias de teste	5
2.1.2	Níveis e Categorias de Teste	6
2.1.3	Geração de Inputs De Teste	8
2.2	Trabalhos Relacionados	10
2.3	Conclusão	11
3	FACT: Arquitetura, Desenvolvimento e Implementação	13
3.1	Ferramentas e Tecnologias Utilizadas	13
3.1.1	Yices e Ocaml Yices	13
3.1.2	Why3	14
3.1.3	CIL	14
3.1.4	Frama-C	14
3.1.5	Crest	15
3.2	Desenho do FACT	17
3.2.1	Execução Concreta	17
3.2.2	Execução Simbólica	17
3.3	Arquitetura do Sistema	19
3.3.1	Estruturas de Dados	19
3.4	Detalhes de Implementação	23
3.4.1	Instrumentação	23
3.4.2	Ligação Entre C e OCaml	33
3.4.3	Execução do FACT	36
3.4.4	Fluxo Genérico de Execução	37
3.4.5	Ciclo de Execução BDFS	38
3.4.6	Incorporação no Frama-C	38
3.5	Conclusão	40
4	Exemplos	41
4.1	Objetivos	41
4.2	Conjunto Testes do Crest	41
4.2.1	Output de Uma Execução <i>Concolic</i>	41
4.2.2	Primeiro Teste	42
4.2.3	Segundo Teste	42
4.2.4	Terceiro Teste	43
4.2.5	Quarto Teste	43
4.2.6	Quinto Teste	44
4.3	Teste de Dimensões Representativas	46

4.4 Conclusão	50
5 Conclusão	53
5.1 Reflexão Crítica e Trabalho Futuro	53
Bibliografia	55
A Manual de Utilização	59
A.1 Requisitos	59
A.2 Compilação e Instalação	59
A.3 Utilização	60
A.4 Documentação	61
B Exemplo Completo de Código Instrumentado	63
C Conjunto de Testes do Crest	73
C.1 Primeiro Teste	73
C.2 Segundo Teste	74
C.3 Terceiro Teste	75
C.4 Quarto Teste	76
C.5 Quinto Teste	76

Lista de Figuras

3.1	Estrutura do Crest.	15
3.2	Nova Estrutura do <i>FACT</i>	19
3.3	Diagrama das Principais Classes.	20
3.4	Fluxo de Execução do <i>FACT</i>	37
4.1	Outputs do segundo teste para o <i>FACT</i>	44
4.2	Outputs do segundo teste para o <i>Crest</i>	45
4.3	Resultados de cobertura obtidos para a leitura de ficheiro e navegação entre menus.	48
4.4	Resultados de cobertura obtidos para a leitura de ficheiro, navegação entre menus e funções de impressão.	49
4.5	Exemplo de execução do teste <i>Concolic</i>	49
4.6	Resultados de cobertura obtidos para a leitura de ficheiro, navegação entre menus, funções de impressão e infrações.	50
4.7	Resultados de cobertura obtidos para o programa completo.	50
A.1	Opções disponíveis para a execução do <i>FACT</i>	61

PREVIEW

Lista de Tabelas

2.1	Tabela comparativa das diferentes ferramentas de teste <i>Concolic</i>	11
4.1	Resultado do <i>Crest</i> para o primeiro teste.	42
4.2	Resultados do <i>FACT</i> para o primeiro teste.	42
4.3	Resultados do <i>FACT</i> para o segundo teste.	43
4.4	Resultados do <i>FACT</i> para o terceiro teste.	43
4.5	Resultados do <i>FACT</i> para o quarto teste.	44
4.6	Resultados do <i>FACT</i> para o quinto teste.	45

PREVIEW

PREVIEW

Listings

2.1	Programa com ramo condicional.	8
3.1	Exemplo de restrições do caminho de execução.	18
3.2	Simplificação de referências de memória complexas.	23
3.3	Resultado da simplificação de referências de memória complexas.	24
3.4	Simplificação de uma instrução for.	24
3.5	Resultado da simplificação de uma instrução for.	24
3.6	Simplificação de uma instrução switch.	25
3.7	Resultado da simplificação de uma instrução switch.	25
3.8	Simplificação de uma instrução While.	26
3.9	Simplificação de uma instrução Do While.	26
3.10	Resolução de um simplificação de uma instrução While.	27
3.11	Resolução da simplificação de uma instrução do While.	27
3.12	Simplificação para apenas um ponto de retorno.	28
3.13	Resultado da simplificação para apenas um ponto de retorno.	28
3.14	Adição de um else a uma instrução if.	29
3.15	Resultado da adição de um else a uma instrução if.	29
3.16	Simplificação de expressões condicionais.	29
3.17	Resultado da simplificação de expressões condicionais.	29
3.18	Formato dos dados do ficheiro “branches”.	30
3.19	Instrumentação da expressão “a*b > 3+c”.	32
3.20	Instrumentação da expressão de uma instrução if.	32
3.21	Instrumentação de uma instrução de atribuição.	32
3.22	Instrumentação do retorno de uma função com argumento.	33
3.23	Instrumentação de uma função que soma dois argumentos.	33
3.24	Inicialização do <i>OCaml runtime</i>	34
3.25	Definição de uma função no OCaml.	35
3.26	Pesquisa da função declarada no OCaml.	35
3.27	Processamento dos argumentos para o Ocaml.	35
3.28	Definição da <i>interface</i> para a função “__Load”.	36
4.1	Comando para Executar o FACT.	41
4.2	Exemplo de leitura dos dados simbólicos.	47
B.1	Exemplo de programa completo em linguagem C.	63
B.2	Exemplo de programa completamente instrumentado.	65
C.1	Pesquisa de elementos num vector.	73
C.2	Utilização de uma variável simbólica em funções.	74
C.3	Programa com expressões matemáticas complexas.	75
C.4	Programa com operações de shift.	76
C.5	Programa com índices de <i>arrays</i> simbólicos	77

PREVIEW

Lista de Acrónimos

API	Application Programming Interface
AST	Abstract Syntax Tree
CFG	Control Flow Graph
CIL	C Intermediate Language
CUTE	Concolic Unit Testing Engine
DART	Directed Automated Random Testing
DFS	Depth First Search
EXE	EXecution generated Executions
FACT	FrAmac Concolic Testing
Ocaml	Objective Caml
(REL)EASE	RELIable And SEcure Computation Group
RWSet	Read-Write Set
SAGE	Scalable Automated Guided Execution
SMART	Systematic Modular Automated Random Testing
UBI	Universidade da Beira Interior

PREVIEW

Capítulo 1

Introdução

O teste de *software* é a principal estratégia usada para encontrar *bugs* de *software*. Normalmente é feito manualmente e representa aproximadamente 40-50% [Som10] do preço final do desenvolvimento do *software*. Ainda assim, este está sujeito a erros, é caro e certamente não é exaustivo. No entanto, *software* sem testes acarreta ainda mais custos, sendo uma necessidade constante a correção de erros.

Uma das principais técnicas usadas no teste de *software* é o teste aleatório, pois é rápido, é fácil gerar *inputs* aleatórios e muitas vezes são formas de atingir *corner cases*, isto é, casos que normalmente são esquecidos e podem causar problemas. A dificuldade surge em gerar estes *inputs* problemáticos, isto é, *inputs* que possam originar problemas durante a execução. Outro problema que surge quando da utilização de teste aleatório é que normalmente não são alcançados todos os ramos do programa, pois muitos dos *inputs* gerados vão originar o mesmo caminho de execução. Existe ainda outro problema associado ao teste aleatório, mais concretamente ao teste de caixa preta, que será discutido no capítulo 2. Para este tipo de teste é necessário um oráculo, um sistema que saiba as respostas corretas para cada *input* gerado e, deste modo, consiga verificar, ou não, a correção do programa.

Para resolver o problema da descoberta de caminhos, surgiram as técnicas de execução simbólica, em que um programa é executado usando variáveis simbólicas em vez de valores concretos. Deste modo é possível descobrir os diversos caminhos de execução possíveis e a partir destes gerar valores concretos para percorrer os novos caminhos descobertos.

Novamente estas técnicas também têm os seus problemas, nomeadamente a eficácia do *Satisfiability Modulo Theories (SMT) Solvers* que resolvem as restrições dos caminhos de execução. Quando a combinação de caminhos é enorme, torna-se computacionalmente dispendioso e inviável resolver as restrições e continuar a execução.

O teste *Concolic* realiza execução simbólica e concreta simultaneamente. A grande vantagem deste tipo de teste sobre execução puramente simbólica é a existência de valores concretos que podem ser usados para refletir sobre estruturas de dados complexas assim como simplificar restrições quando estas já não estão ao alcance do *SMT*.

1.1 Motivação e Objetivos

O principal objetivo desta dissertação é a construção de uma aplicação que permita realizar testes *Concolic* para um programa desenvolvido em Linguagem C e a respetiva integração em *Frama-C* [frab].

A grande vantagem de incorporar esta aplicação no *Frama-C* é a possibilidade de permitir que o programador tenha todo um leque de possibilidades para analisar o seu *software* numa única ferramenta que, atualmente, já é utilizada por grandes empresas, facilitando também o pro-

cesso de divulgação desta ferramenta.

Este projeto pode ser visto como a continuação do projeto de licenciatura intitulado *TESTA - Geração Automática de Testes Unitários* [Per13], embora este ainda estivesse numa fase embrionária de desenvolvimento.

É ainda importante referir que este trabalho foi desenvolvido em parceria entre o grupo de investigação (REL)EASE (RELIABLE And SEcure Computation Group) da Universidade da Beira Interior (UBI) e a empresa Educad no âmbito do projeto PROVA. Este trabalho foi financiado por uma bolsa de investigação e espera-se que posteriormente seja incorporado no PROVA.

Os objetivos previstos no plano de trabalho são os seguintes:

- Revisão da literatura sobre a engenharia dos testes de *software*;
- Estudo e análise de ferramentas de teste *Concolic*;
- Escolha de uma ferramenta capaz de realizar testes *Concolic* e respetivo estudo do seu funcionamento;
- Estudo do desenvolvimento de *plugins* para *Frama-C*;
- Desenho de uma solução de teste *Concolic* para *Frama-C*;
- Desenvolvimento e implementação da ferramenta de teste *Concolic* com base nos princípios na ferramenta estudada;
- Incorporação da ferramenta desenvolvida no *Frama-C*;
- Realização de um caso de estudo que comprove o correto funcionamento da ferramenta desenvolvida.

1.2 Contribuições

Dado o estado atual das capacidades das ferramentas de execução de teste *Concolic*, não foram feitas grandes contribuições ao estado da arte destas no âmbito desta dissertação. Mas considera-se uma etapa importante de entender e conseguir utilizar os conceitos atualmente existentes para construir uma base robusta e funcional que, eventualmente, permita a continuação e melhoria no âmbito deste tipo de teste.

Por outro lado, a principal contribuição, além de utilizar estes conceitos com sucesso numa ferramenta funcional, o *FACT*, esta foi implementada em *Ocaml* que, por sua vez, abre um leque enorme de possibilidades para o seu crescimento. Um dos fatores que diretamente resultou desta escolha foi a integração do *FACT* em *Frama-C* que possibilita outras análises para além do teste *Concolic*. No contexto da Engenharia de teste, no nosso entender, tal contribuição tem a sua relevância!

Tendo como ponto de partida o *FACT*, torna-se possível testar e implementar novas estratégias de pesquisa, métodos de execução, expandir para outros *SMT Solvers* através do *Why3* ou até mesmo utilizar informação de outros *plug-ins* do *Frama-C* em prol do teste *Concolic*.